

Yum: preference-shaped media walls for taste memory 1

Alan N. Pham 2

AO Labs 3

yum.aolabs.io 4

Yum is not a food-only website. It is a personal taste-memory wall for cars, food, alcohol, 5
and K-pop girls. The system combines a static media archive, source-aware image metadata, 6
category balancing, remote candidate discovery, optional model-assisted ranking, and cross- 7
device preference memory into a wall that opens directly into images instead of a dashboard. 8
The design problem is therefore broader than serving attractive media: Yum must preserve 9
taste, avoid obvious repeats, keep loading when services are unavailable, remember what has 10
been hidden, and make four different subject areas feel like one coherent visual surface. 11

1 Introduction

Most media walls are either static collections or generic recommendation feeds. Static collections preserve intent, but they become repetitive. Generic feeds keep moving, but they usually lose the specific taste contract that made the page worth opening in the first place. Yum is designed for the middle ground: a personally shaped wall that starts from known-good material, keeps extending through candidate discovery, and remembers negative feedback across sessions.

The subject scope is intentionally explicit. Yum is for cars, food, alcohol, and K-pop girls. Treating the project as a food site misses the actual product shape. The value is in allowing those categories to coexist without one category taking over, without source repeats becoming obvious, and without the page becoming a control panel before it becomes a visual experience.

The public surface is a static GitHub Pages site at `yum.aolabs.io`. A Node service provides optional dynamic behavior: preference persistence, model-assisted curation, and K-pop candidate discovery. The page still functions when those services are not available, because the first requirement is that the wall opens quickly and remains visually useful.

2 Design Requirements

2.1 Media-first entry

The first screen should be the wall itself. The interface avoids a heavy onboarding flow, filter dashboard, or explanatory landing page. Captions, source links, and controls are available, but they should not compete with the images.

2.2 Four-category taste contract

The feed must hold four subject areas at once: cars, food, alcohol, and K-pop girls. This requires a stricter contract than appending images to a masonry grid. The system needs category balance, source de-duplication, person and car-group spacing, and fallback material that still matches the intended taste when online sources are weak.

2.3 Preference memory

Yum uses hide actions as meaningful taste signals. Hidden source keys remove exact items. Hidden samples become stronger negative examples. Nearby visible items can become weaker positive examples because they were present at the time of the rejection and were not removed. This keeps the stored memory compact while still giving the ranking layer useful direction.

3 System Architecture

Yum has three main layers: a public static wall, a small backend service, and a preference model that is mirrored between browser state and remote storage.

Layer	Implementation role
Public surface	GitHub Pages serves the wall, static assets, preview metadata, icon assets, and the paper link at <code>yum.aolabs.io</code> .
Backend service	A Node service exposes <code>/api/curate</code> , <code>/api/preferences</code> , and <code>/api/kpop-candidates</code> .
AI ranking	The curation endpoint can score image candidates using the configured OpenAI model and the current taste memory.
Persistence	Preference state is stored locally for immediate response and remotely for cross-device continuity.
Fallback behavior	Static seed pools and generated candidate templates keep the wall useful when network sources or model calls fail.

Table 1. Current Yum architecture. The system is intentionally static-first, with dynamic curation layered on top instead of required for initial rendering.

3.1 Public wall

The public wall is responsible for presentation: responsive masonry placement, tile shapes, source links, captions, visual grouping, lazy loading, and interaction state. Each tile carries more information than its visible image: a category, shape, source key, caption, source URL, focus hint, and sometimes person or group metadata.

3.2 Backend service

The backend preserves the parts of the system that cannot be solved with a static page alone. It accepts and returns preference state, filters and ranks curation candidates, caches responses, and exposes K-pop candidate retrieval as a separate endpoint. This keeps the wall operational while still allowing the taste model to improve beyond a fixed bundle.

4 Feed Contract

The feed contract is the heart of Yum. The wall should feel like one visual surface, but the categories have different failure modes.

1. Cars need modern, tasteful reference images and should avoid stale show-floor, traffic, SUV, old-car, or disliked model/color patterns.
2. Food needs cravings, memory, restaurants, meals, dishes, screenshots, and the kind of image that is worth saving for later.
3. Alcohol needs clean table, bottle, drink, and nightlife-adjacent material without turning the whole feed into bar stock imagery.
4. K-pop girl imagery needs to stay softer and more personal, avoiding heavy stage energy, noisy editorial frames, and low-quality portraits.

This contract is enforced by multiple mechanisms rather than one filter. Static pools guarantee a baseline. Blocked-term checks remove obvious misses. Source keys prevent exact repeats. Visual groups and person groups prevent one look from dominating the scroll. Recent placement history keeps the wall from forming blocks of the same category.

5 Curation and Preference Stack

Yum separates candidate generation from candidate selection. Candidate generation answers the question, “what could be shown next?” Candidate selection answers the harder question, “what should be shown next, given the current wall and what has already been rejected?”

Mechanism	Purpose
Static seed pools	Start the wall with known-good cars, food, alcohol, and K-pop girl media.
Blocked terms	Reject low-fit candidates before they reach the wall.
Hidden keys	Remove exact source items that the user has rejected.
Hidden samples	Store stronger negative examples for model-assisted ranking.
Kept samples	Preserve lightweight positive context from nearby non-hidden items.
Model ranking	Rank candidates against current taste rules and preference memory when the API is configured.
Source rotation	Keep the wall moving when one candidate source runs thin or stale.

Table 2. Curation mechanisms used to keep Yum from collapsing into a generic image feed.

6 Persistence

73

The browser stores a local mirror of preference state so the interface can react immediately. The backend stores the same class of compact state in `preferences.json`: hidden keys, hidden samples, kept samples, a version number, and an update timestamp. This is enough to make the wall feel persistent across devices without storing a full behavioral history.

74

75

76

77

The persistence model favors small, explicit signals. This is a practical choice. A large opaque log would be harder to reason about and harder to repair. A compact preference object can be inspected, capped, synced, and applied directly to candidate selection.

78

79

80

7 Paper and Site Placement

81

The paper appears at the top of the public Yum site, before the wall, with two consistent actions: open the paper and return to the wall. This matches the AO Labs convention that projects with real technical work should show the paper clearly instead of hiding it below the experience.

82

83

84

The paper itself is generated from a LaTeX source so it can share formatting with the existing AO Labs paper style: NewTX text and math fonts, line numbers, compact scientific geometry, colored section rules, ragged-right captions, and a restrained header/footer system.

85

86

87

8 Limits and Next Work

88

Yum is already more than a static gallery, but several limits remain.

89

-
1. Remote curation quality depends on the configured model and the quality of available candidates.
 2. Preference memory is intentionally compact, so it captures directional taste better than long written explanations.
 3. Static fallback pools need ongoing maintenance because the desired scope is specific and changes with taste.
 4. The wall should expose enough system state to be trustworthy without turning the first screen into an operations dashboard.

9 Conclusion

Yum is a preference-shaped media wall for cars, food, alcohol, and K-pop girls. Its core contribution is not the presence of those categories alone, but the operational system around them: static-first rendering, source-aware metadata, category balance, candidate ranking, compact preference memory, and cross-device persistence. Those pieces allow a simple visual wall to behave like a durable taste surface instead of a one-time image collection.